

Project #2 - MongoDB / Document Stores

Student: `Charilaos Ntourakis`

StudentID: `t8220210`

Course: `Big Data Management Systems`

1. Domain Description and Schema

This report documents the design and implementation of the **ShopNest** MongoDB database, a simplified multi-vendor e-commerce marketplace. The solution models product listings, customers, and orders while also introducing a custom `vendors` collection to support seller-level analytics. The complete implementation is included in `Proj2\_Mongo\_solution.js`.

Collections used

- * `vendors`
- * `products`
- * `customers`
- * `orders`

1.1 Schema summary

The project keeps the required collections and fields from the assignment and adds a small custom schema for vendors.

`vendors`

```
{
  _id: ObjectId,
  name: String,
  email: String,
  country: String,
  joined_at: Date
}
```

`products`

```
{
  _id: ObjectId,
  sku: String,
  name: String,
  category: String,
  vendor_id: ObjectId,
  price: NumberDecimal,
  stock: NumberInt,
  ratings: {
    average: Number,
    count: NumberInt
  },
}
```

```

    tags: [String],
    created_at: Date
}

```

`customers`

```

{
  _id: ObjectId,
  email: String,
  name: String,
  address: {
    city: String,
    country: String
  },
  tier: String,
  joined_at: Date
}

```

`orders`

```

{
  _id: ObjectId,
  customer_id: ObjectId,
  status: String,
  items: [
    {
      product_id: ObjectId,
      sku: String,
      name: String,
      qty: NumberInt,
      unit_price: NumberDecimal
    }
  ],
  total: NumberDecimal,
  created_at: Date
}

```

1.2 Embedding decision justification

Embedding order line items inside each `orders` document is the right choice for this workload because order reads are expected to be much more frequent than item-level writes after checkout. Typical application flows such as the order detail page, customer order history, and admin shipment view need the full order together with all purchased items in one request, so embedding reduces joins and improves read latency. Write amplification is limited because order line items are mostly immutable after order creation. The maximum document size is still safe in this model because typical e-commerce orders contain a relatively small number of products and remain far below MongoDB's 16 MB document limit. Referencing would be preferable in an extreme scenario such as very large B2B orders with

thousands of items or when individual line items must be updated independently after checkout.

2. Populating / Updating

2.1 Indexes created

The script creates the required indexes before inserting data and explains them with comments:

- * Unique index on `products.sku`
- * Compound index on `orders(customer\_id, created\_at DESC)` for customer order-history lookups
- * Index on `orders.status` for dashboard filtering
- * Text index on `products(name, tags, category)` to support full-text search

2.2 Seed data summary

The script inserts at least the minimum required dataset:

- * 4 vendors
- * 20 seed products across 5 categories
- * 10 customers with mixed tiers and countries
- * 30 orders referencing valid customer and product ids
- * 6 cancelled orders
- * 1 customer with 6 orders

After the duplicate-insert demonstration, the collection contains 21 product documents in total because `SNT-9999` is inserted once and skipped the second time.

2.3 Conditional insert with duplicate detection

Implemented function:

```
function insertProductSafe(product) {
  const previous = db.products.findOneAndUpdate(
    { sku: product.sku },
    { $setOnInsert: product },
    { upsert: true, returnDocument: "before" }
  );

  if (previous === null) {
    print("Inserted: " + product.sku);
  } else {
    print("Skipped (exists): " + product.sku);
  }
}
```

Demonstration calls:

```
insertProductSafe(duplicateTestProduct);
insertProductSafe({ ...duplicateTestProduct, name: "Test Widget Duplicate" });
```

Observed behavior:

- * First call inserts `SNT-9999`
- * Second call skips the duplicate SKU

2.4 Bulk price update

Implemented with one `updateMany` call:

```
const discountResult = db.products.updateMany(
  { category: "Electronics", stock: { $gt: 50 } },
  { $mul: { price: NumberDecimal("0.90") } }
);
```

Observed result:

- * `matchedCount = 5`
- * `modifiedCount = 5`

3. Queries

For every query below, the report includes:

- * the query code
- * the first 3 result documents in pretty-printed Extended JSON
- * a one-sentence English description

3.1 Simple field filter + projection

```
db.products
  .find(
    {
      category: "Books",
      "ratings.average": { $gte: 4.0 },
      stock: { $gt: 0 }
    },
    { _id: 0, name: 1, price: 1, "ratings.average": 1, tags: 1 }
  )
  .sort({ price: 1 });
```

Description: Returns in-stock books with average rating at least 4.0, projecting only selected fields and ordering by lowest price first.

First 3 result documents:

[

```

{
  "name": "Sci-Fi Anthology 2026",
  "price": {
    "$numberDecimal": "18.90"
  },
  "ratings": {
    "average": 4.1
  },
  "tags": [
    "fiction",
    "sale"
  ]
},
{
  "name": "Modern Greek Cooking",
  "price": {
    "$numberDecimal": "22.50"
  },
  "ratings": {
    "average": 4.4
  },
  "tags": [
    "cooking",
    "new-arrival"
  ]
},
{
  "name": "Data Modeling with MongoDB",
  "price": {
    "$numberDecimal": "34.00"
  },
  "ratings": {
    "average": 4.7
  },
  "tags": [
    "database",
    "featured"
  ]
}
]

```

3.2 Array query operators (`\$all` and `\$in`)

Query A:

```
db.products.find(
  { tags: { $all: ["new-arrival", "featured"] } },
  { _id: 0, sku: 1, name: 1, tags: 1 }
);
```

Description: Returns products that contain both `new-arrival` and `featured` tags.

First 3 result documents:

```
[
  {
    "sku": "SNT-1001",
    "name": "Wireless Headphones Pro",
    "tags": [
      "audio",
      "wireless",
      "featured",
      "new-arrival"
    ]
  },
  {
    "sku": "SNT-1010",
    "name": "Clean Code Patterns",
    "tags": [
      "programming",
      "featured",
      "new-arrival"
    ]
  },
  {
    "sku": "SNT-1013",
    "name": "Running T-Shirt",
    "tags": [
      "sports",
      "new-arrival",
      "featured"
    ]
  }
]
```

Query B:

```
db.products.find(
```

```

    { tags: { $in: ["clearance", "sale"] } } },
    { _id: 0, sku: 1, name: 1, tags: 1 }
  );

```

Description: Returns products tagged with either `clearance` or `sale`.

First 3 result documents:

```

[
  {
    "sku": "SNT-1002",
    "name": "Wireless Earbuds Lite",
    "tags": [
      "audio",
      "wireless",
      "sale"
    ]
  },
  {
    "sku": "SNT-1005",
    "name": "Bluetooth Speaker Mini",
    "tags": [
      "audio",
      "clearance"
    ]
  },
  {
    "sku": "SNT-1006",
    "name": "USB-C Charger 65W",
    "tags": [
      "accessory",
      "sale",
      "featured"
    ]
  }
]

```

3.3 Full-text search

```

db.products.find(
  { $text: { $search: "wireless headphones" } } },
  { score: { $meta: "textScore" }, name: 1, price: 1, _id: 0 }
).sort({ score: { $meta: "textScore" } });

```

Description: Returns text-matching products for the phrase "wireless headphones", ranked by relevance score.

Only 2 matches were found in this dataset.

Result documents:

```
[
  {
    "score": {
      "$numberDecimal": "4.00"
    },
    "name": "Wireless Headphones Pro",
    "price": {
      "$numberDecimal": "80.910"
    }
  },
  {
    "score": {
      "$numberDecimal": "1.50"
    },
    "name": "Wireless Earbuds Lite",
    "price": {
      "$numberDecimal": "44.910"
    }
  }
]
```

3.4 Nested document query

```
db.customers.find(
  {
    "address.country": { $in: ["Greece", "Germany"] },
    tier: "gold"
  },
  { _id: 0, name: 1, email: 1, address: 1 }
);
```

Description: Returns gold-tier customers who live in Greece or Germany.

Only 2 matches were found in this dataset.

Result documents:

```
[
```

```
{
  "name": "Alice Papadopoulou",
  "email": "alice.papadopoulou@example.com",
  "address": {
    "city": "Athens",
    "country": "Greece"
  }
},
{
  "name": "Elena Kraus",
  "email": "elena.kraus@example.com",
  "address": {
    "city": "Munich",
    "country": "Germany"
  }
}
]
```

3.5 Query on embedded arrays (`\$elemMatch`)

```
db.orders.find(
  { items: { $elemMatch: { qty: { $gt: 3 } } } },
  { _id: 1, customer_id: 1, status: 1, items: 1, created_at: 1 }
);
```

Description: Returns orders that include at least one line item with quantity greater than 3.

First 3 result documents:

```
[
  {
    "_id": {
      "$oid": "663000000000000000000000000002"
    },
    "customer_id": {
      "$oid": "661000000000000000000000000001"
    },
    "status": "shipped",
    "items": [
      {
        "product_id": {
          "$oid": "66200000000000000000000000000a"
        },
        "sku": "SNT-1010",
```

```
    "name": "Clean Code Patterns",
    "qty": 1,
    "unit_price": {
      "$numberDecimal": "41.20"
    }
  },
  {
    "product_id": {
      "$oid": "662000000000000000000000f"
    },
    "sku": "SNT-1015",
    "name": "Cotton Socks 5-Pack",
    "qty": 4,
    "unit_price": {
      "$numberDecimal": "12.99"
    }
  }
],
"created_at": {
  "$date": "2026-01-25T10:00:00Z"
}
},
{
  "_id": {
    "$oid": "663000000000000000000000d"
  },
  "customer_id": {
    "$oid": "6610000000000000000000003"
  },
  "status": "shipped",
  "items": [
    {
      "product_id": {
        "$oid": "662000000000000000000000f"
      },
      "sku": "SNT-1015",
      "name": "Cotton Socks 5-Pack",
      "qty": 6,
      "unit_price": {
        "$numberDecimal": "12.99"
      }
    }
  ]
},
"created_at": {
```

```

    "$date": "2026-03-30T10:00:00Z"
  }
},
{
  "_id": {
    "$oid": "663000000000000000000000000013"
  },
  "customer_id": {
    "$oid": "661000000000000000000000000005"
  },
  "status": "pending",
  "items": [
    {
      "product_id": {
        "$oid": "662000000000000000000000000012"
      },
      "sku": "SNT-1018",
      "name": "Desk Lamp Minimal",
      "qty": 1,
      "unit_price": {
        "$numberDecimal": "33.30"
      }
    },
    {
      "product_id": {
        "$oid": "66200000000000000000000000000f"
      },
      "sku": "SNT-1015",
      "name": "Cotton Socks 5-Pack",
      "qty": 5,
      "unit_price": {
        "$numberDecimal": "12.99"
      }
    }
  ],
  "created_at": {
    "$date": "2026-03-14T10:00:00Z"
  }
}
]

```

3.6 Date range query (`\$gte` / `\$lte`)

```

db.orders.find(
{

```

```

    status: "delivered",
    created_at: {
      $gte: ISODate("2026-03-19T00:00:00Z"),
      $lte: ISODate("2026-04-18T00:00:00Z")
    }
  },
  { _id: 1, customer_id: 1, status: 1, total: 1, created_at: 1 }
).sort({ created_at: -1 });

```

Description: Returns delivered orders within the last 30 days of the seed-data timeline, newest first.

First 3 result documents:

```

[
  {
    "_id": {
      "$oid": "66300000000000000000000000000001c"
    },
    "customer_id": {
      "$oid": "66100000000000000000000000000009"
    },
    "status": "delivered",
    "total": {
      "$numberDecimal": "108.40"
    },
    "created_at": {
      "$date": "2026-04-15T10:00:00Z"
    }
  },
  {
    "_id": {
      "$oid": "663000000000000000000000000000014"
    },
    "customer_id": {
      "$oid": "66100000000000000000000000000005"
    },
    "status": "delivered",
    "total": {
      "$numberDecimal": "144.40"
    },
    "created_at": {
      "$date": "2026-04-12T10:00:00Z"
    }
  },

```

```

{
  "_id": {
    "$oid": "663000000000000000000000000011"
  },
  "customer_id": {
    "$oid": "661000000000000000000000000004"
  },
  "status": "delivered",
  "total": {
    "$numberDecimal": "224.90"
  },
  "created_at": {
    "$date": "2026-04-10T10:00:00Z"
  }
}
]

```

4. Aggregation

4.1 Revenue by category

Code:

```

db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $unwind: "$items" },
  {
    $lookup: {
      from: "products",
      localField: "items.product_id",
      foreignField: "_id",
      as: "product"
    }
  },
  { $unwind: "$product" },
  {
    $group: {
      _id: "$product.category",
      total_revenue: {
        $sum: { $multiply: ["$items.unit_price", "$items.qty"] }
      },
      product_set: { $addToSet: "$items.product_id" }
    }
  },
  {
    $project: {

```

```

        _id: 0,
        category: "$_id",
        total_revenue: 1,
        products_sold: { $size: "$product_set" }
    }
},
{ $sort: { total_revenue: -1 } }
]);

```

Description: Computes delivered-order revenue per product category and the number of distinct products sold in each category.

First 3 result documents:

```

[
  {
    "category": "Electronics",
    "total_revenue": {
      "$numberDecimal": "916.30"
    },
    "products_sold": 5
  },
  {
    "category": "Clothing",
    "total_revenue": {
      "$numberDecimal": "344.00"
    },
    "products_sold": 3
  },
  {
    "category": "Home",
    "total_revenue": {
      "$numberDecimal": "332.10"
    },
    "products_sold": 3
  }
]

```

4.2 Customer lifetime value

Code:

```

db.orders.aggregate([
  {

```

```

$group: {
  _id: "$customer_id",
  total_orders: { $sum: 1 },
  total_spent: {
    $sum: {
      $cond: [
        { $eq: ["$status", "delivered"] },
        "$total",
        NumberDecimal("0")
      ]
    }
  },
  avg_order_value: { $avg: "$total" }
}
},
{
  $lookup: {
    from: "orders",
    let: { cid: "$_id" },
    pipeline: [
      {
        $match: {
          $expr: {
            $and: [
              { $eq: ["$customer_id", "$$cid"] },
              { $eq: ["$status", "delivered"] }
            ]
          }
        }
      }
    ],
    { $unwind: "$items" },
    {
      $lookup: {
        from: "products",
        localField: "items.product_id",
        foreignField: "_id",
        as: "p"
      }
    },
    { $unwind: "$p" },
    {
      $group: {
        _id: "$p.category",
        units: { $sum: "$items.qty" }
      }
    }
  }
}

```

```

        }
    },
    { $sort: { units: -1, _id: 1 } },
    { $limit: 1 }
],
as: "fav"
}
},
{
  $lookup: {
    from: "customers",
    localField: "_id",
    foreignField: "_id",
    as: "customer"
  }
},
{
  $unwind: {
    path: "$customer",
    preserveNullAndEmptyArrays: true
  }
},
{
  $project: {
    _id: 0,
    customer_id: "$_id",
    customer_name: { $ifNull: ["$customer.name", null] },
    total_orders: 1,
    total_spent: 1,
    avg_order_value: 1,
    favourite_category: {
      $ifNull: [{ $arrayElemAt: ["$fav._id", 0] }, null]
    }
  }
},
{ $sort: { total_spent: -1 } },
{ $limit: 10 }
]);

```

Description: Computes total order count, delivered spending, average order value, and favourite delivered category for each customer, then joins customer names via `\$lookup` and returns the top 10 customers by spending.

First 3 result documents:

```
[
  {
    "customer_id": {
      "$oid": "661000000000000000000000000001"
    },
    "customer_name": "Alice Papadopoulou",
    "total_orders": 6,
    "total_spent": {
      "$numberDecimal": "364.40"
    },
    "avg_order_value": {
      "$numberDecimal": "129.41"
    },
    "favourite_category": "Electronics"
  },
  {
    "customer_id": {
      "$oid": "661000000000000000000000000004"
    },
    "customer_name": "Nikos Galanis",
    "total_orders": 3,
    "total_spent": {
      "$numberDecimal": "331.40"
    },
    "avg_order_value": {
      "$numberDecimal": "140.37"
    },
    "favourite_category": "Clothing"
  },
  {
    "customer_id": {
      "$oid": "661000000000000000000000000002"
    },
    "customer_name": "Markus Weber",
    "total_orders": 4,
    "total_spent": {
      "$numberDecimal": "293.80"
    },
    "avg_order_value": {
      "$numberDecimal": "142.10"
    },
    "favourite_category": "Electronics"
  }
]
```

4.3 Monthly sales trend with `\$bucket`

Code:

```
const monthBoundaries = [
  ISODate("2026-01-01T00:00:00Z"),
  ISODate("2026-02-01T00:00:00Z"),
  ISODate("2026-03-01T00:00:00Z"),
  ISODate("2026-04-01T00:00:00Z"),
  ISODate("2026-05-01T00:00:00Z")
];

db.orders.aggregate([
  {
    $bucket: {
      groupBy: "$created_at",
      boundaries: monthBoundaries,
      default: "outside_window",
      output: {
        total_revenue: { $sum: "$total" },
        number_of_orders: { $sum: 1 },
        avg_order_value: { $avg: "$total" }
      }
    }
  },
  { $match: { _id: { $ne: "outside_window" } } },
  {
    $project: {
      _id: 0,
      month: { $dateToString: { format: "%Y-%m", date: "$_id" } },
      total_revenue: 1,
      number_of_orders: 1,
      avg_order_value: 1
    }
  },
  { $sort: { month: 1 } }
]);
```

Description: Groups orders into month buckets, labels each bucket with `\$dateToString`, and computes total revenue, order count, and average order value per month.

Monthly output:

[

```

{
  "month": "2026-01",
  "total_revenue": {
    "$numberDecimal": "697.53"
  },
  "number_of_orders": 6,
  "avg_order_value": {
    "$numberDecimal": "116.26"
  }
},
{
  "month": "2026-02",
  "total_revenue": {
    "$numberDecimal": "755.50"
  },
  "number_of_orders": 7,
  "avg_order_value": {
    "$numberDecimal": "107.93"
  }
},
{
  "month": "2026-03",
  "total_revenue": {
    "$numberDecimal": "1302.79"
  },
  "number_of_orders": 10,
  "avg_order_value": {
    "$numberDecimal": "130.28"
  }
},
{
  "month": "2026-04",
  "total_revenue": {
    "$numberDecimal": "1209.30"
  },
  "number_of_orders": 7,
  "avg_order_value": {
    "$numberDecimal": "172.76"
  }
}
]

```

Bar chart:

```

2026-01 | ##### 697.53
2026-02 | ##### 755.50
2026-03 | ##### 1302.79
2026-04 | ##### 1209.30

```

The chart shows a gradual increase from January to February, a clear peak in March, and a small drop in April while remaining above January and February.

4.4 Vendor performance report (`\$lookup` + `\$group`)

Code:

```

db.vendors.aggregate([
  {
    $lookup: {
      from: "products",
      localField: "_id",
      foreignField: "vendor_id",
      as: "catalog"
    }
  },
  {
    $addFields: {
      total_products_listed: { $size: "$catalog" }
    }
  },
  {
    $lookup: {
      from: "orders",
      let: { vendorId: "$_id" },
      pipeline: [
        { $match: { status: "delivered" } },
        { $unwind: "$items" },
        {
          $lookup: {
            from: "products",
            localField: "items.product_id",
            foreignField: "_id",
            as: "product"
          }
        },
        { $unwind: "$product" },
        { $match: { $expr: { $eq: ["$product.vendor_id", "$$vendorId"] } } }
      ]
    }
  },
  {
    $group: {

```

```

        _id: "$items.product_id",
        product_name: { $first: "$items.name" },
        units: { $sum: "$items.qty" },
        revenue: {
            $sum: { $multiply: ["$items.unit_price", "$items.qty"] }
        }
    }
},
],
as: "sales_by_product"
}
},
{
    $unwind: {
        path: "$sales_by_product",
        preserveNullAndEmptyArrays: true
    }
},
{
    $sort: {
        _id: 1,
        "sales_by_product.units": -1,
        "sales_by_product.product_name": 1
    }
},
{
    $group: {
        _id: "$_id",
        vendor_name: { $first: "$name" },
        vendor_email: { $first: "$email" },
        total_products_listed: { $first: "$total_products_listed" },
        total_units_sold: {
            $sum: { $ifNull: ["$sales_by_product.units", 0] }
        },
        gross_revenue: {
            $sum: {
                $ifNull: ["$sales_by_product.revenue", NumberDecimal("0")]
            }
        },
        top_product: { $first: "$sales_by_product.product_name" }
    }
},
{
    $project: {

```

```

    _id: 0,
    vendor_name: 1,
    vendor_email: 1,
    total_products_listed: 1,
    total_units_sold: 1,
    gross_revenue: 1,
    top_product: { $ifNull: ["$top_product", null] }
  }
},
{ $sort: { gross_revenue: -1 } }
]);

```

Description: Joins vendors with their catalog and delivered-order sales to compute catalog size, sold units, gross revenue, and each vendor's best-selling product.

First 3 result documents:

```

[
  {
    "vendor_name": "Aegean Gadgets",
    "vendor_email": "ops@aegeangadgets.com",
    "total_products_listed": 7,
    "total_units_sold": 15,
    "gross_revenue": {
      "$numberDecimal": "916.30"
    },
    "top_product": "USB-C Charger 65W"
  },
  {
    "vendor_name": "UrbanFit Apparel",
    "vendor_email": "sales@urbanfitapparel.com",
    "total_products_listed": 6,
    "total_units_sold": 16,
    "gross_revenue": {
      "$numberDecimal": "672.50"
    },
    "top_product": "Running T-Shirt"
  },
  {
    "vendor_name": "HomeCraft Living",
    "vendor_email": "team@homecraftliving.fr",
    "total_products_listed": 3,
    "total_units_sold": 4,
    "gross_revenue": {

```

```
        "$numberDecimal": "332.10"
    },
    "top_product": "Air Fryer XL"
}
]
```

5. How to Run

Run the solution as a script file from `mongosh`:

```
mongosh Proj2_Mongo_solution.js > run_output.txt
```

You can also run it without redirecting the output:

```
mongosh Proj2_Mongo_solution.js
```

The script:

- drops and recreates the `shopnest` database
- creates indexes before insertion
- inserts seed data
- demonstrates duplicate-safe insertion
- applies the bulk discount update
- prints the first results for all required queries and aggregations

6. Submission Files

Final submission files:

- `Proj2\_Mongo\_solution.js`
- `Proj2\_Mongo\_report.pdf`